Semantic Autonomous Indoor Navigation Robot

CPG No. 305
Harish Singla (102303301)
Jalaj Gupta (102303925)
Naveen Goyal (102303757)
Sparsh Gupta (102353019)
Yatharth Misra (102303304)

Project Overview and Need Analysis

The Indoor Navigation Gap

GPS-enabled routing systems excel in outdoor environments but fail indoors due to signal attenuation and lack of satellite visibility.

Industries—hospitals, warehouses, manufacturing facilities—require autonomous systems that perceive obstacles and understand their semantic meaning for safe, intelligent navigation.

Real-World Limitations

Only 6% of warehouses actually use AGVs (G2 report) because magnetic tapes are expensive to install and impossible to change quickly.

In hospitals, delivery robots like Moxi were discontinued at facilities like MultiCare Health System (early 2026) because nurses reported they 'got in the way' and required constant human intervention.

Related Work and Literature Survey

1

Basic LiDAR SLAM Systems

Construct occupancy grids using laser scanning but lack semantic understanding. Robots detect obstacles without identifying objects.

2

Cloud-Based Visual Navigation

Offload image processing to remote servers for semantic segmentation. Introduces high network latency incompatible with real-time navigation.

3

Standard AGV Implementations

Automated Guided Vehicles follow fixed magnetic or visual paths. Inflexible routing prevents adaptation to dynamic environments.

Research Gap: Fusing edge-AI semantic vision with NoSQL data pipelines for real-time indoor mapping with affordable, low-latency hardware.

Problem Statement and Objectives

Problem

Absence of affordable, low-latency, semantically-aware indoor navigation systems that fuse real-time perception with intelligent path planning.

01

Autonomous Indoor Mapping System

Develop SLAM-based navigation using 2D LiDAR for real-time occupancy grid construction and localization

02

Edge-AI Semantic Segmentation

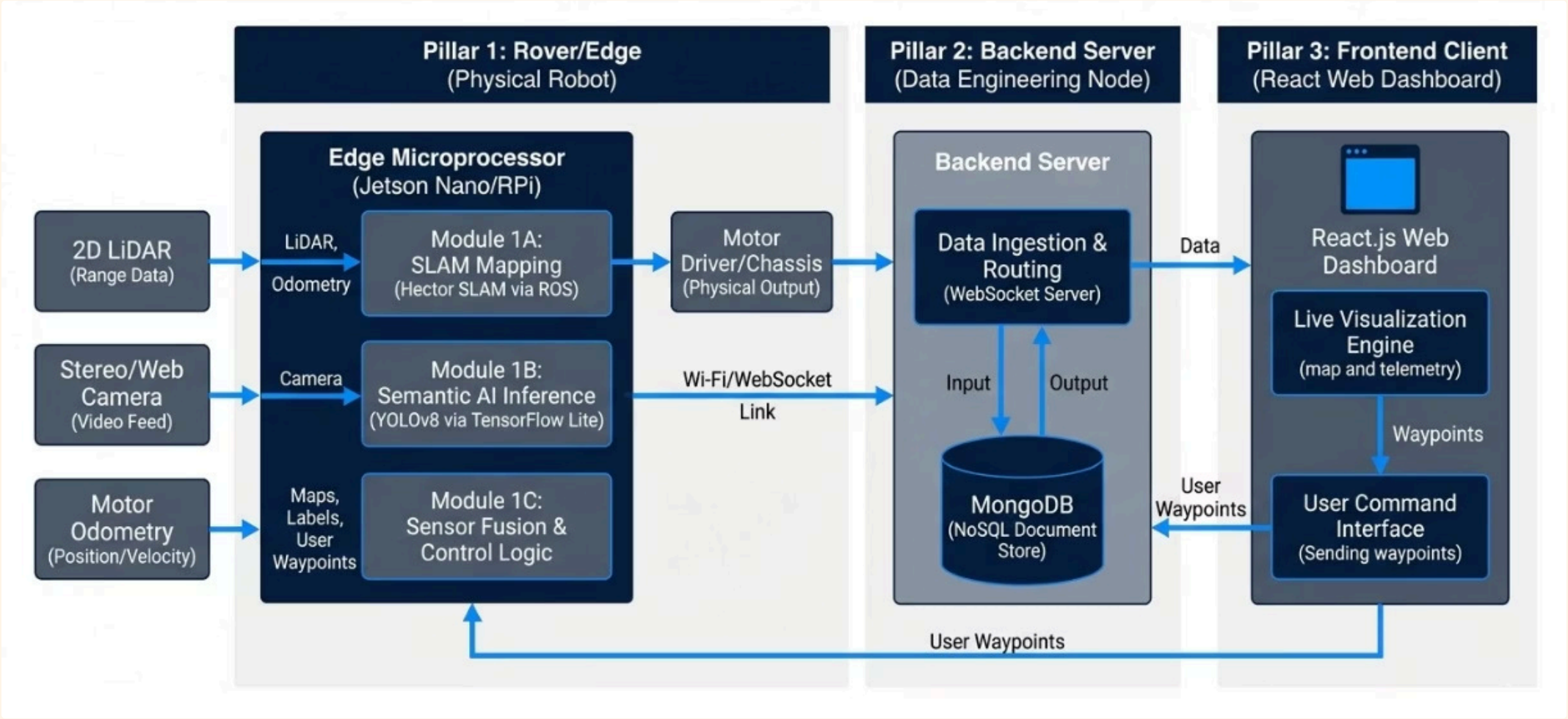
Integrate lightweight object detection models for real-time identification and classification of indoor objects.

03

Full-Stack Dashboard

Build React-based interface for live telemetry visualization, waypoint programming, and system monitoring

System Architecture



Assumptions and Constraints

Assumptions

- Stable Wi-Fi or local network infrastructure for telemetry transmission
- Relatively flat indoor terrain with minimal elevation changes
- Moderate lighting conditions suitable for camera-based perception

Constraints

- Edge-device compute limits balancing processing power against frame rate requirements
- Sensor range limitations of affordable 2D LiDAR modules
- Power budget restrictions for mobile autonomous operation

Project Execution Plan

Broad Methodology

Hardware Integration

Assemble motor chassis, sensors (LiDAR, stereo camera), and compute platform (Jetson Nano/Raspberry Pi)

ROS Implementation

Deploy Robot Operating System for SLAM-based localization and mapping using 2D LiDAR data

Edge-AI Deployment

Implement lightweight object detection models (YOLO architecture) for real-time semantic segmentation

Full-Stack Development

Build MongoDB backend for spatial data and React frontend for live dashboard visualization.

Project Requirements

Hardware

- Jetson Nano or Raspberry Pi 4
- 2D LiDAR sensor (RPLIDAR A1/A2)
- Stereo or high-resolution webcam
- Motorized chassis with encoder feedback

Software Stack

- Robot Operating System (ROS)
- Python 3 for sensor fusion
- TensorFlow Lite / PyTorch Mobile
- MongoDB for spatial data

Frontend Tools

- React.js with Material UI
- WebSocket for real-time telemetry
- Map visualization libraries

Project Outcomes

Functional Autonomous Rover Prototype

Mobile platform integrating SLAM-based localization with semantic object recognition. Capable of dynamic path planning around identified obstacles and navigating to programmed waypoints using fused sensor data.

Real-Time Web Dashboard

Interactive interface displaying live occupancy maps, identified object classifications, sensor telemetry, and battery status. Supports remote waypoint programming and mission control through WebSocket connections.

Work Plan

6-Month Timeline

1

Months 1-2

Hardware assembly and integration; basic SLAM implementation; ROS node development

2

Months 3-4

Edge-AI model training and deployment; semantic segmentation integration; sensor fusion

3

Months 5-6

Backend pipeline development; React dashboard implementation; system testing and validation

Role Distribution

Hardware & ROS Lead

Sensor integration, motor control, SLAM implementation, and low-level ROS node development

Computer Vision Engineer

YOLO model training, semantic segmentation, object detection, and real-time inference optimization

Data Engineering & Backend Lead

MongoDB schema design, WebSocket server, telemetry pipeline, and spatial data management

Frontend & UI/UX Developer

React dashboard implementation, map visualization, user interface design, and responsive layout

References:

https://dl.acm.org/doi/pdf/10.1145/3542945

https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=7219438

https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=8578485